



CS Penguins Teacher Workbook

Hands-on activities, questions and answers for every lesson

Name: _____

Class: _____

Date: _____

How to use this workbook

One page per lesson. First watch the lesson on CS Penguins (about 10 minutes), then do the activity together, then let students answer the questions. The answers are at the end.

Suggested 45-minute lesson: 10 min watch and discuss the lesson, 15 min hands-on activity, 10 min practice quiz on CS Penguins, 10 min questions from this page.

CS Penguins needs no student accounts. Students can use any device with a browser. Progress stays on each device.

Lessons in this workbook

1.	Binary Numbers	Ages 7-10
2.	Inside the Computer	Ages 7-11
3.	Loops	Ages 7-10
4.	Variables	Ages 7-10
5.	Data Types	Ages 9-13
6.	If and Else	Ages 7-10
7.	Robot Penguin	Ages 7-10
8.	Flowcharts and Algorithms	Ages 8-12
9.	Find the Bug	Ages 8-12
10.	Arrays	Ages 9-13
11.	Functions	Ages 10-14
12.	Logic Gates	Ages 11-15
13.	Files and Folders	Ages 7-11
14.	Spreadsheets	Ages 9-13
15.	Databases	Ages 11-15
16.	Passwords and Phishing	Ages 8-13
17.	Digital Citizenship	Ages 7-12
18.	Breadth-First Search	Ages 8-12
19.	Depth-First Search	Ages 8-12
20.	Bubble Sort	Ages 8-12
21.	Selection Sort	Ages 8-12
22.	Insertion Sort	Ages 8-12
23.	Binary Search	Ages 9-13



24.	Letters and Pictures as Numbers	Ages 9-13
25.	Speed of Algorithms	Ages 11-15
26.	Secret Messages	Ages 9-13
27.	How the Internet Sends a Message	Ages 11-15
28.	Data and Charts	Ages 9-13
29.	Teach the Computer	Ages 11-15
30.	Recursion	Ages 12-15
31.	Topological Sort	Ages 11-15
32.	Stack and Queue	Ages 9-13
33.	Heap	Ages 11-15
34.	Dijkstra's Algorithm	Ages 11-15
35.	Bellman-Ford	Ages 12-15
36.	Prim's Algorithm	Ages 11-15
37.	Kruskal's Algorithm	Ages 11-15
38.	Quick Sort	Ages 11-15
39.	Merge Sort	Ages 11-15



1. Binary Numbers

Ages 7-10

Today we learn

Write a number using only 1s and 0s, the way a computer does, by turning switches on and off.

Do it together (no computer needed)

Make four cards: 8, 4, 2 and 1. A card face up means 1, face down means 0. Ask students to show today's date, or their age, with the cards.

Questions

1. What is the number 6 in binary (cards 8, 4, 2, 1)?

2. What number is 1011?

3. Why do computers only use 0 and 1?



2. Inside the Computer

Ages 7-11

Today we learn

Follow a letter and a drawing through the five parts of a computer.

Do it together (no computer needed)

Make five signs: Input, CPU, Memory, Storage and Output. Walk through "type a letter" and "save a drawing" together, and let students stand at the sign where each thing happens.

Questions

1. Which part is the keyboard?

2. Which part forgets everything when the power goes off?

3. Where does a saved picture stay for tomorrow?



3. Loops

Ages 7-10

Today we learn

Use a loop to add up the numbers from 1 to n, one round at a time.

Do it together (no computer needed)

Play "Repeat 10 times": the class says "clap and take one step" and counts the rounds out loud. Then ask: how would you tell a robot to do this without writing it 10 times?

Questions

1. A loop adds $1 + 2 + 3 + 4$. What is the total?

2. What happens if a loop never reaches its stopping point?

3. Why is a loop better than writing 100 separate lines?



4. Variables

Ages 7-10

Today we learn

Follow a small program line by line and always know what is inside each box.

Do it together (no computer needed)

Use three cups called a, b and temp. Put 1 coin in a and 3 coins in b. Ask students to swap the coins between a and b, using temp as the spare cup.

Questions

1. $x = 4$, then $x = x + 3$. What is in x?

2. $a = 2$, then $b = a$, then $a = 5$. What is in b?

3. Why does swapping two boxes need a third box?



5. Data Types

Ages 9-13

Today we learn

Sort values into number, text and true or false, and see how the same + works differently on each.

Do it together (no computer needed)

Put cards with 7, "7", hello, 3.5, true and "cat" on the board. Students sort them into three bins: numbers, text and true/false. Discuss why "7" in quotes is text.

Questions

1. What is $3 + 4$ for numbers, and $"3" + "4"$ for text?

2. Which type is true?

3. Why does the computer need types?



6. If and Else

Ages 7-10

Today we learn

Follow a program that asks true-or-false questions and chooses what to do.

Do it together (no computer needed)

Play "If you ...": call out rules such as "If your name has an A, stand up. Otherwise, sit down." Change the rule and play again, then add a second rule with "otherwise if".

Questions

1. The temperature is 6. Which branch runs: $\text{temp} < 0$, $\text{temp} < 10$ or else?

2. What does ELSE do?

3. Can two branches of the same if run in one go?



7. Robot Penguin

Ages 7-10

Today we learn

Follow a program of commands, step by step, and see where the robot penguin ends up.

Do it together (no computer needed)

Mark a 5 x 5 grid on the floor with tape and hide a paper fish. One student is the robot and may only follow the commands "forward", "turn right" and "turn left". The class writes the program on the board first, then the robot runs it exactly.

Questions

1. The penguin looks north and gets "turn right", then "forward". Which way does it move?

2. What does "repeat 3 times: forward" do?

3. The robot ends one square short of the fish. What do you do?



8. Flowcharts and Algorithms

Ages 8-12

Today we learn

Read a flowchart: follow the arrows, do the steps and answer the questions to see how an algorithm works.

Do it together (no computer needed)

Write the steps for making a jam sandwich on cards, in a mixed-up order. Pairs put them in order and then draw the steps as a flowchart. Add a question diamond: "Is there jam left?"

Questions

1. What does an oval mean in a flowchart?

2. What does a diamond mean?

3. Why do we draw an algorithm before programming it?



9. Find the Bug

Ages 8-12

Today we learn

Run a program by hand, notice that the answer is wrong, and find the broken line.

Do it together (no computer needed)

Give students a short recipe or set of instructions with one wrong step (for example, a robot walk that turns the wrong way). They follow it with a pencil, find the mistake and fix it.

Questions

1. What is a bug?

2. A program should give 6 but gives 5. What do you do first?

3. Why should you not change lines at random?



10. Arrays

Ages 9-13

Today we learn

Read and change the boxes of an array using their index, then use a loop to add them up.

Do it together (no computer needed)

Line up 5 students with numbers 0 to 4 on their shirts (stickers). Call out "read box 3" or "write 9 into box 1". Then make a 3 x 3 grid of students for a table.

Questions

1. An array has 6 boxes. What is the index of the last one?

2. `a = [4, 8, 2]`. What is `a[1]`?

3. What does `table[2][0]` mean?



11. Functions

Ages 10-14

Today we learn

Call functions with different inputs and follow how they return results.

Do it together (no computer needed)

Be a "function machine": the student behind a desk follows a rule (for example "double it"). The class calls in numbers and the machine answers. Then change the rule and play again.

Questions

1. $\text{double}(x) = x * 2$. What is $\text{double}(7)$?

2. $\text{area}(w, h) = w * h$. What is $\text{area}(3, 5)$?

3. Why write a function instead of repeating the steps?



12. Logic Gates

Ages 11-15

Today we learn

Try every combination of switches on a logic gate and fill in its truth table.

Do it together (no computer needed)

Two students hold hands to show AND: the lamp is on only when both are holding hands up. Then try OR (at least one up) and NOT (the opposite). Fill in the truth table together.

Questions

1. AND: A is 1 and B is 0. What is the lamp?

2. OR: A is 1 and B is 0. What is the lamp?

3. NOT: the input is 0. What is the lamp?



13. Files and Folders

Ages 7-11

Today we learn

Walk down a tree of folders and write the path to a file.

Do it together (no computer needed)

Draw a folder tree of a school bag on the board (Bag, Books, Maths, Art...). Students write the path to each item. Then let them design a tidy folder tree for their own computer or tablet.

Questions

1. What is the path of homework.txt in School/Maths?

2. Can two files share a name?

3. What is the top folder called?



14. Spreadsheets

Ages 9-13

Today we learn

Use cell names and formulas, and watch a sum update by itself.

Do it together (no computer needed)

Use a paper grid. Students write their own numbers (for example the pocket money they saved each week) in column A, then write the formula for the total in a cell and recalculate by hand when one number changes.

Questions

1. Which cell is in column C, row 4?

2. A1 is 3 and B1 is 5. What does `=A1+B1` show?

3. A1:A3 holds 2, 4 and 6. What does `=SUM(A1:A3)` show?



15. Databases

Ages 11-15

Today we learn

Filter, sort and count the rows of a penguin table.

Do it together (no computer needed)

Collect facts about the class on cards (name, age, favourite animal). Treat them as a table. Filter ("favourite animal is penguin"), count and sort by age with real cards.

Questions

1. What is one row of a table called?

2. What does a filter do?

3. Sorting by age, smallest first: who comes first?



16. Passwords and Phishing

Ages 8-13

Today we learn

Check passwords with five tests and find the clues in a trick message.

Do it together (no computer needed)

Students invent a long made-up password sentence on paper, then check it against the five tests. Then give pairs a pretend message and let them circle the clues. Remind everyone never to use a real password in class.

Questions

1. Name two things that make a password strong.

2. A message says "act in 5 minutes and send your password". What do you do?

3. Is it fine to share a password with a best friend?



17. Digital Citizenship

Ages 7-12

Today we learn

Practise safe, kind and honest choices online.

Do it together (no computer needed)

Act out short scenes (a stranger asks where you live, a friend wants to copy text, a rude comment). The class decides together what a good digital citizen does and why.

Questions

1. Name two kinds of personal information to keep private.

2. You want to use a photo from the internet. What do you check and do?

3. Name two clues that a website can be trusted.



18. Breadth-First Search

Ages 8-12

Today we learn

Find the shortest way to the fish by checking all the close places first.

Do it together (no computer needed)

Draw a map of 8 school rooms on the board. Pass a ball outwards like ripples: first the neighbours of the start, then their neighbours. Write the order on the board.

Questions

1. Which waiting list does BFS use?

2. What does BFS find on a map with equal roads?

3. Which places does BFS visit right after the start?



19. Depth-First Search

Ages 8-12

Today we learn

Find a way to the fish by following one path as far as it goes.

Do it together (no computer needed)

Tape a small maze on the floor. A student walks it by going as deep as possible, and walking back to the last junction at every dead end.

Questions

1. Which waiting list does DFS use?

2. What does DFS do at a dead end?

3. Does DFS always find the shortest way?



20. Bubble Sort

Ages 8-12

Today we learn

Put the penguins in order from the shortest to the tallest by swapping neighbours.

Do it together (no computer needed)

Line up 6 students in random order of height. Compare neighbours in pairs, from the left, and swap if the left one is taller. Repeat the walk until nobody swaps.

Questions

1. After one full walk, where is the tallest penguin?

2. When can bubble sort stop?

3. Is bubble sort fast for long lines?



21. Selection Sort

Ages 8-12

Today we learn

Put the penguins in order by always choosing the shortest one that is left and moving it to the front.

Do it together (no computer needed)

Hand out 6 cards with numbers. In each round, find the smallest card that is left and swap it into the first open place. Ask: how many places are fixed after each round?

Questions

1. How many places are fixed after 3 rounds?

2. What does selection sort search for in every round?

3. Does it get faster if the line is almost sorted?



22. Insertion Sort

Ages 8-12

Today we learn

Put the penguins in order by taking them one at a time and sliding each one back to its right place.

Do it together (no computer needed)

Give each student a few playing cards. Sort them by inserting one new card at a time into the right place of the cards already in order.

Questions

1. What is the sorted part at the very start?

2. When is insertion sort very fast?

3. When does a penguin stop moving left?



23. Binary Search

Ages 9-13

Today we learn

Find one penguin in a line that is already in order, by looking at as few penguins as possible.

Do it together (no computer needed)

Play "guess my number" from 1 to 100. Always guess the middle and say higher or lower. How many guesses do you need at most? (About 7.)

Questions

1. Must the line be sorted before binary search?

2. We look for 8. The middle penguin is 5. Which half do we keep?

3. About how many looks do 1,000 penguins need?



24. Letters and Pictures as Numbers

Ages 9-13

Today we learn

See how a computer stores a word and a picture using nothing but numbers.

Do it together (no computer needed)

Give every student the alphabet with numbers (A = 1 ... Z = 26). They write their name as numbers, swap with a partner and decode each other. Then draw a 5 x 5 grid, colour in a simple picture and write each row as 0s and 1s.

Questions

1. What number does the letter D get if A = 1?

2. In a black and white picture, what do 1 and 0 mean?

3. Write the row "dark, light, dark" as numbers.



25. Speed of Algorithms

Ages 11-15

Today we learn

Watch three penguins race and discover how the number of steps grows when the pile gets bigger.

Do it together (no computer needed)

Race with cards: n students hold a number. The Walker checks every number, the Waddler checks every pair, the Slider repeatedly halves the group. Count the steps out loud for 8 and 16 students.

Questions

1. A pile has 8 items. How many steps does the Walker need?

2. A pile has 8 items. How many steps does the Waddler need?

3. A pile has 16 items. How many times can the Slider halve it?



26. Secret Messages

Ages 9-13

Today we learn

Turn a word into a secret code by moving each letter along the alphabet, and learn why secrets need to be strong.

Do it together (no computer needed)

Make two paper alphabet wheels, one inside the other. Turn the inner one by 3 and let pairs write short secret messages for each other, then decode them by turning the wheel back.

Questions

1. With a shift of 3, what does C become?

2. With a shift of 3, what does Y become?

3. Why is this code not safe for real secrets?



27. How the Internet Sends a Message

Ages 11-15

Today we learn

Follow a message that is cut into packets, travels in pieces and is put back together.

Do it together (no computer needed)

Write a sentence on a strip of paper and cut it into pieces of 3 letters. Number each piece, mix them up, pass them to a partner and let them rebuild the sentence. Remove one piece and see how the partner notices.

Questions

1. Why does every packet get a number?

2. A message has 12 letters and each packet holds 4. How many packets?

3. What happens when a packet gets lost?



28. Data and Charts

Ages 9-13

Today we learn

Turn a table of numbers into a bar chart and find the biggest, the smallest, the total and the average.

Do it together (no computer needed)

Ask the class how many siblings, pets or books they have. Draw a bar chart on the board together, then find the biggest, the smallest, the total and the average.

Questions

1. What does a taller bar mean?

2. The numbers are 2, 4 and 6. What is the average?

3. Can two different weeks have the same average?



29. Teach the Computer

Ages 11-15

Today we learn

See how a computer can guess the group of a new penguin by learning from examples, which is a simple kind of artificial intelligence.

Do it together (no computer needed)

Draw 6 example animals on the board as dots with two numbers each (size and weight), in two groups. Add a new animal and let students find the nearest example and guess its group. Then discuss: what if our examples were unfair?

Questions

1. What does the computer do with a new penguin?

2. Where do the computer's answers come from?

3. Why can a computer guess wrong?



30. Recursion

Ages 12-15

Today we learn

Follow a function that calls itself, watch the calls pile up, and see the answers come back.

Do it together (no computer needed)

Use nesting boxes or cups of different sizes. Open them one by one, and then say what happens when you reach the smallest one. Then act out `fact(4)`: each student waits for the answer from the next student.

Questions

1. What is the base case of `fact(n)`?

2. What is `fact(4)`?

3. What happens without a base case?



31. Topological Sort

Ages 11-15

Today we learn

Put the places in an order so that every arrow points forward: a place always comes after the places that point to it.

Do it together (no computer needed)

Draw an arrow map for getting ready for school: socks → shoes, shirt → jacket, shirt → backpack. Find an order where every arrow points forward. Is there more than one?

Questions

1. What does an arrow mean?

2. Which places can go first?

3. What breaks topological sort?



32. Stack and Queue

Ages 9-13

Today we learn

See the difference between a queue (a line) and a stack (a pile) when the same penguins arrive and leave.

Do it together (no computer needed)

Use a stack of books and a line of students. Add four items to each (A, B, C, D), then take two out. Who leaves from the line? Which book leaves from the stack?

Questions

1. Who leaves a queue first?

2. Who leaves a stack first?

3. Which one does BFS use, and which does DFS use?



33. Heap

Ages 11-15

Today we learn

Build a family tree of penguins where every parent is shorter than its children, so the shortest is always at the top.

Do it together (no computer needed)

Put 7 sticky notes on the board as a family tree, smallest on top. Add a new note at the bottom and let it climb up, swapping with its parent.

Questions

1. Where is the smallest number in a heap?

2. A new penguin 1 sits under its parent 4. Do they swap?

3. Is a heap completely sorted?



34. Dijkstra's Algorithm

Ages 11-15

Today we learn

Find the cheapest way from the start to the fish when every path has a price.

Do it together (no computer needed)

Draw a map where every road has a cost in minutes. Find the cheapest way from home to school. Then find the way with the fewest roads. Are they the same?

Questions

1. Which place does Dijkstra pick next?

2. What can break Dijkstra's algorithm?

3. Can the cheapest way use more roads than another way?



35. Bellman-Ford

Ages 12-15

Today we learn

Find the cheapest cost to reach every place from the start, on a map with one-way roads where one road gives a discount.

Do it together (no computer needed)

Draw a map with one-way roads and one discount road (cost -2). Go through all the roads in rounds, in the same order, until a round changes nothing.

Questions

1. What can Bellman-Ford do that Dijkstra cannot?

2. When do we stop?

3. How many passes might we need at most?



36. Prim's Algorithm

Ages 11-15

Today we learn

Connect all the places with the cheapest set of paths, growing a tree from the start.

Do it together (no computer needed)

Connect 6 houses to one water pipe using the cheapest pipes. Start at one house and always add the cheapest pipe that reaches a new house.

Questions

1. What does Prim pick in every step?

2. How many paths does a tree over 6 places have?

3. Can a spanning tree contain a loop?



37. Kruskal's Algorithm

Ages 11-15

Today we learn

Connect all the places with the cheapest set of paths by choosing from the whole list.

Do it together (no computer needed)

Write the pipe prices on cards and sort them. Take the cheapest first, and skip any pipe that would join two places that are already connected.

Questions

1. What does Kruskal do first?

2. When do we skip a path?

3. Do Prim and Kruskal find the same total cost?



38. Quick Sort

Ages 11-15

Today we learn

Put the penguins in order from the shortest to the tallest.

Do it together (no computer needed)

Give 6 students numbers. Pick the last one as the pivot. Shorter numbers go to its left, taller to its right. Repeat inside each group.

Questions

1. What is the pivot?

2. After a round, where is the pivot?

3. When is quick sort slow?



39. Merge Sort

Ages 11-15

Today we learn

Sort the penguins by cutting the line in halves again and again, then joining the pieces back in order.

Do it together (no computer needed)

Cut a line of 8 students in halves again and again until each is alone. Then merge pairs back, always comparing the two front students.

Questions

1. What does merge sort do first?

2. When merging two sorted pieces, which penguin do we take?

3. Is merge sort fast even on a sorted line?



Answer key

1. Binary Numbers

- 1) 0110, because $4 + 2 = 6$.
- 2) 11, because $8 + 2 + 1 = 11$.
- 3) A tiny switch is either on or off, so two symbols are enough.

2. Inside the Computer

- 1) Input.
- 2) Memory (RAM).
- 3) In storage.

3. Loops

- 1) 10.
- 2) It repeats forever. This is called an infinite loop.
- 3) It is much shorter and does the same job.

4. Variables

- 1) 7.
- 2) 2. b has its own copy.
- 3) Without a spare box, one of the values would be overwritten and lost.

5. Data Types

- 1) 7 for numbers. For text the pieces are stuck together and give "34".
- 2) True or false (a boolean).
- 3) The type tells it what it may do with a value, for example add numbers or join text.

6. If and Else

- 1) $\text{temp} < 10$, so "wear a hat". 6 is not smaller than 0, but it is smaller than 10.
- 2) It runs only when every question above it was false.
- 3) No. The first true branch is chosen and the others are skipped.

7. Robot Penguin

- 1) East, because turning right makes it look east.
- 2) It moves forward 3 squares. A loop saves writing the same command three times.
- 3) Run the program step by step, find the command that went wrong and fix only that one.

8. Flowcharts and Algorithms

- 1) Start or End.
- 2) A question with a yes arrow and a no arrow.
- 3) It shows the order and the choices clearly, so mistakes are easy to see.

9. Find the Bug

- 1) A mistake in a program.
- 2) Run it line by line and write down the boxes after every line to find where it goes wrong.
- 3) You may break something else. Find the broken line first, then fix it.

10. Arrays

- 1) 5, because counting starts at 0.



- 2) 8.
- 3) Row 2, column 0.

11. Functions

- 1) 14.
- 2) 15.
- 3) You write the recipe once and use it many times.

12. Logic Gates

- 1) 0 (off). AND needs both to be 1.
- 2) 1 (on). OR needs at least one 1.
- 3) 1 (on). NOT flips the input.

13. Files and Folders

- 1) School/Maths/homework.txt, the folder names joined with /.
- 2) Yes, if they are in different folders, because their paths differ.
- 3) The root.

14. Spreadsheets

- 1) C4.
- 2) 8.
- 3) 12.

15. Databases

- 1) A record.
- 2) It keeps only the rows that match a rule.
- 3) The youngest.

16. Passwords and Phishing

- 1) Long, and a mix of capital letters, small letters, numbers and symbols.
- 2) Do not click or reply, and show a grown-up.
- 3) No. A password stays secret.

17. Digital Citizenship

- 1) For example: address, school, phone number, passwords.
- 2) Check the licence and write who made it.
- 3) It names the author and the date, and says where the facts come from.

18. Breadth-First Search

- 1) A queue (first in, first out).
- 2) The way with the fewest steps.
- 3) All the neighbours of the start, before anything further away.

19. Depth-First Search

- 1) A stack (last in, first out).
- 2) It goes back to the last place that still has an unvisited path.
- 3) No. It finds a way, but not always the shortest.

20. Bubble Sort

- 1) At the end of the line.



- 2) When a whole walk makes no swaps.
- 3) No. It needs about $n \times n$ steps.

21. Selection Sort

- 1) 3.
- 2) The smallest penguin that is left.
- 3) No. It still searches the whole rest of the line every round.

22. Insertion Sort

- 1) The first penguin on its own.
- 2) When the line is already, or almost, sorted.
- 3) When its left neighbour is shorter, or it reaches the front.

23. Binary Search

- 1) Yes. Otherwise the answers are wrong.
- 2) The taller half.
- 3) About 10.

24. Letters and Pictures as Numbers

- 1) 4.
- 2) 1 is a dark pixel and 0 is a light pixel.
- 3) 101.

25. Speed of Algorithms

- 1) 8.
- 2) 64, because $8 \times 8 = 64$.
- 3) 4 times, because $2 \times 2 \times 2 \times 2 = 16$.

26. Secret Messages

- 1) F.
- 2) B, because after Z the alphabet starts again at A.
- 3) There are only 25 shifts, so anyone can try them all.

27. How the Internet Sends a Message

- 1) So the receiver can put the packets back in the right order and see which one is missing.
- 2) 3.
- 3) The receiver asks the sender to send that packet again.

28. Data and Charts

- 1) A bigger number.
- 2) 4, because 12 divided by 3 is 4.
- 3) Yes. That is why we also look at the chart.

29. Teach the Computer

- 1) It finds the nearest example and copies its group.
- 2) From the examples we gave it.
- 3) If the examples are unfair or missing, it copies their mistakes.

30. Recursion

- 1) $\text{fact}(1) = 1$. It stops the calling.



- 2) 24, because $4 \times 3 \times 2 \times 1 = 24$.
- 3) The calls never stop and the program crashes.

31. Topological Sort

- 1) "This must come first."
- 2) Places with no arrows pointing into them.
- 3) A circle of arrows, because nobody can go first.

32. Stack and Queue

- 1) The one who arrived first.
- 2) The one who arrived last (it is on top).
- 3) BFS uses a queue and DFS uses a stack.

33. Heap

- 1) At the top.
- 2) Yes, because a parent must be the smaller one.
- 3) No. Only every parent is smaller than its children.

34. Dijkstra's Algorithm

- 1) The cheapest place that is not finished yet.
- 2) Roads with a negative cost.
- 3) Yes. Several cheap roads can cost less than one expensive road.

35. Bellman-Ford

- 1) Handle roads with negative costs.
- 2) When a whole pass over the roads changes nothing.
- 3) About the number of places minus 1.

36. Prim's Algorithm

- 1) The cheapest path that leaves the tree.
- 2) 5.
- 3) No.

37. Kruskal's Algorithm

- 1) It sorts all the paths from cheapest to most expensive.
- 2) When it would make a loop, because its two ends are already connected.
- 3) Yes, both find the cheapest possible tree.

38. Quick Sort

- 1) The "boss" penguin used to split the group.
- 2) In its final place.
- 3) When the pivot is always the smallest or tallest, for example a sorted line with the last penguin as pivot.

39. Merge Sort

- 1) It cuts the line in halves until every piece has one penguin.
- 2) The shorter of the two front penguins.
- 3) Yes. It always takes about $n \times \log_2(n)$ steps.